

tf.compat.v1.nn.convolution Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn/convolution

Computes sums of N-D convolutions (actually cross-correlation).

Function Signatures

```
tf.compat.v1.nn.convolution( input, filter, padding,  
    strides=None, dilation_rate=None, name=None,  
    data_format=None, filters=None, dilations=None )
```

Code Examples

```
tf.compat.v1.nn.convolution(  
    input,  
    filter,  
    padding,  
    strides=None,  
    dilation_rate=None,  
    name=None,  
    data_format=None,  
    filters=None,  
    dilations=None  
)
```

```
tf.compat.v1.nn.convolution(
input,
filter,
padding,
strides=None,
dilation_rate=None,
name=None,
data_format=None,
filters=None,
dilations=None
)
```

```
output[b, x[0], ..., x[N-1], k] =
sum_{z[0], ..., z[N-1], q}
filter[z[0], ..., z[N-1], q, k] *
padded_input[b,
x[0]*strides[0] + dilation_rate[0]*z[0],
...,
x[N-1]*strides[N-1] + dilation_rate[N-1]*z[N-1],
q]
```

```
output[b, x[0], ..., x[N-1], k] =
sum_{z[0], ..., z[N-1], q}
filter[z[0], ..., z[N-1], q, k] *
padded_input[b,
x[0]*strides[0] + dilation_rate[0]*z[0],
...,
x[N-1]*strides[N-1] + dilation_rate[N-1]*z[N-1],
q]
```

```
convolution(input, data_format, **kwargs) =
tf.transpose(convolution(tf.transpose(input, [0] + range(2,N+2) +
[1])),
**kwargs),
[0, N+1] + range(1, N+1))
```

```
convolution(input, data_format, **kwargs) =  
tf.transpose(convolution(tf.transpose(input, [0] + range(2,N+2) +  
[1]),  
**kwargs),  
[0, N+1] + range(1, N+1))
```

```
`[batch_size] + output_spatial_shape + [out_channels]`
```

```
`[batch_size] + output_spatial_shape + [out_channels]`
```

Documentation

Computes sums of N-D convolutions (actually cross-correlation).

This also supports either output striding via the optional strides parameter or atrous convolution (also known as convolution with holes or dilated convolution, based on the French word "trous" meaning holes in English) via the optional dilation_rate parameter. Currently, however, output striding is not supported for atrous convolutions.

Specifically, in the case that data_format does not start with "NC", given a rank (N+2) input Tensor of shape

```
[num_batches,  
input_spatial_shape[0],  
...,  
input_spatial_shape[N-1],  
num_input_channels],
```

a rank (N+2) filter Tensor of shape

```
[spatial_filter_shape[0],  
...,  
spatial_filter_shape[N-1],  
num_input_channels,  
num_output_channels],
```

an optional `dilation_rate` tensor of shape `N` (defaults to `[1]*N`) specifying the filter upsampling/input downsampling rate, and an optional list of `N` strides (defaults to `[1]*N`), this computes for each N-D spatial output position (`x[0]`, ..., `x[N-1]`):

where `b` is the index into the batch, `k` is the output channel number, `q` is the input channel number, and `z` is the N-D spatial offset within the filter. Here, `padded_input` is obtained by zero padding the input using an effective spatial filter shape of $(\text{spatial_filter_shape}-1) * \text{dilation_rate} + 1$ and output striding strides.

In the case that `data_format` does start with "NC", the input and output (but not the filter) are simply transposed as follows:

It is required that $1 \leq N \leq 3$.

Returns

if `data_format` is `None` or does not start with "NC", or

if `data_format` starts with "NC",

where `output_spatial_shape` depends on the value of padding.

If padding == "SAME":

output_spatial_shape[i] = ceil(input_spatial_shape[i] / strides[i])

If padding == "VALID":

output_spatial_shape[i] =

ceil((input_spatial_shape[i] -

(spatial_filter_shape[i]-1) * dilation_rate[i])

/ strides[i]).

Raises